

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS
ORGANIZACIÓN DE LENGUAJES Y
COMPILADORES 1
PRIMER SEMESTRE 2026
PROYECTO 2
Andy Jefferson González Fuentes 202231338



USAC
TRICENTENARIA
Universidad de San Carlos de Guatemala

MANUAL TÉCNICO

Tokens del analizador léxico:

Palabras reservadas

EXTENDS → "extends"
AT_FOR → "@for"
FROM → "from"
THROUGH → "through"
TO → "to"

Propiedades simples

P_HEIGHT → "height"
P_WIDTH → "width"
P_COLOR → "color"
P_PADDING → "padding"
P_MARGIN → "margin"
P_BORDER → "border"

Palabras reservadas (keywords)

KW_COMPONENT → "component"
KW_FUNCTION → "function"
KW_INT → "int"
KW_FLOAT_T → "float"
KW_STRING_T → "string"
KW_BOOLEAN_T → "boolean"
KW_CHAR_T → "char"
KW_IF → "if"
KW_ELSE → "else"
KW_SWITCH → "Switch" (nota: mayúscula inicial según spec)
KW_CASE → "case"
KW_DEFAULT → "default"

KW_FOR → "for"
KW_EACH → "each"
KW_TRACK → "track"
KW_EMPTY → "empty"
KW_WHILE → "while"
KW_DO → "do"
KW_T → "T"
KW_IMG → "IMG"
KW_FORM → "FORM"
KW_INPUT_TEXT → "INPUT_TEXT"
KW_INPUT_NUMBER → "INPUT_NUMBER"
KW_INPUT_BOOL → "INPUT_BOOL"
KW_SUBMIT → "SUBMIT"

Palabras reservadas y

KW_IMPORT → import
KW_EXECUTE → execute
KW_INT → int
KW_FLOAT_T → float
KW_STRING_T → string
KW_BOOLEAN_T → boolean
KW_CHAR_T → char
KW_LET → let
KW_CONST → const
KW_VAR → var
KW_FUNCTION → function
KW_MAIN → main
KW_RETURN → return
KW_BREAK → break
KW_CONTINUE → continue
KW_LOAD → load
KW_IF → if
KW_ELSE → else
KW_FOR → for
KW_WHILE → while
KW_DO → do
KW_SWITCH → switch
KW_CASE → case
KW_DEFAULT → default

Token	Lexema	Contexto de uso
NO_TERMINAL_KW	No_Terminal	Declarar un no terminal en Syntax
INITIAL_SIM_KW	Initial_Sim	Declarar el símbolo inicial en Syntax
TERMINAL_KW	Terminal	Declarar un terminal en Lex
SYNTAX	Syntax	Encabezado del bloque sintáctico
LEX	Lex	Encabezado del bloque léxico

- **Nombres de terminales y no terminales:**

TERMINAL_NAME	\\$_[a-zA-Z][a-zA-Z0-9]*	\$_Numero, \$_FIN, \$_P_Ab
NT_NAME	\%_[a-zA-Z][a-zA-Z0-9]*	%_S, %_Prod_A, %_Expr
NT_NAME	\%[a-zA-Z][a-zA-Z0-9]*	%S (tolerancia de errores)

La segunda regla para NT_NAME (sin guion bajo) es una regla de tolerancia: si el usuario escribe %S en lugar de %_S, el parser no falla con un error léxico incomprensible. Ambas variantes producen el mismo token NT_NAME.

- **Operadores y separadores:**

Token	Lexema	Uso
ARROW	<-	Asigna una expresión regular a un terminal
PROD_ARROW	<=	Define el cuerpo de una producción
PIPE		Separador de alternativas en una producción
SEMICOLON	;	Fin de declaración (terminal, NT o producción)
STAR	*	Operador Kleene: cero o más repeticiones
PLUS	+	Cerradura positiva: una o más repeticiones
QUESTION	?	Operador opcional: cero o una vez
LPAREN	(	Agrupación para concatenación de expresiones
RPAREN	)	Cierre de agrupación

- **Expresiones regulares en bloques lex:**

Token	Patrón	Ejemplo
RANGE_ALPHA	"[aA-zZ]"	Terminal \$_Letra <- [aA-zZ] ;
RANGE_DIGIT	"[0-9]"	Terminal \$_Num <- [0-9] ;
LITERAL	'texto'	Terminal \$_FIN <- 'FIN' ;

Los rangos [aA-zZ] y [0-9] son los únicos rangos permitidos en Wison. Se definen como strings literales exactos en el lexer, lo que significa que [A-Z] o [a-z] solos generarían un error sintáctico.

- **Comentarios y espacios:**

Tipo	Patrón	Comportamiento
Comentario de línea	# hasta fin de línea	Ignorado completamente
Comentario de bloque	/** ... */	Ignorado completamente
Espacios y saltos	\s+ (incluye Unicode)	Ignorados completamente

Gramática BNF del lenguaje Wison

A continuación se presenta la gramática completa en notación BNF que describe la estructura del lenguaje Wison, tal como está implementada en wison.json:

```
import_decl
  : KW_IMPORT STR SEMI
  {
    checkImportExtension($2, @1.first_line,
@1.first_column+1);
    checkImportDuplicate($2, @1.first_line,
@1.first_column+1);
    $$ = { type:'Import', path:$2, line:@1.first_line,
col:@1.first_column+1 };
  }
  ;

function_decl
  : fn_sig LBRACE stmt_list RBRACE
  {
    _fnDepth--;
    exitScope();
    $$ = { type:'FunctionDecl', name:$1.name,
params:$1.params, body:$3,
          line:$1.line, col:$1.col };
  }
  ;
```

```

main_decl
: KW_MAIN main_open LBRACE stmt_list RBRACE
  {
    checkMainUnique(@1.first_line, @1.first_column+1);
    _fnDepth--;
    exitScope();
    $$ = { type:'Main', body:$4, line:@1.first_line,
col:@1.first_column+1 };
  } ;

para styles

/* Valor de color: nombre CSS o variable */
color_val
: IDENT { $$ = makeColor($1); }
| VAR { $$ = makeVar($1); }
;

/* Dimensión: número, porcentaje o expresión */
dimension
: expr { $$ = $1; }
| PERCENT { $$ = makePercent($1); }
;

* Declaración de estilo */
/*
Formas válidas:
mi-estilo { ... }
mi-estilo extends super-estilo { ... }
my-font-$i { ... } ← dentro de un @for
*/

style_rule
: style_name LBRACE property_list RBRACE
  { $$ = makeStyle($1, null, $3, loc(@1)); }

| style_name EXTENDS style_name LBRACE property_list RBRACE

```

```

        { $$ = makeStyle($1, $3, $5, loc(@1)); }

| style_name LBRACE property_list error
{
    syntaxErrors.push({
        lexema:      '}',
        linea:       @$.last_line,
        columna:     @$.last_column,
        descripcion: 'Se esperaba "]" para cerrar la
declaración de estilo "' + $1 + '".'
    });
    $$ = makeStyle($1, null, $3, loc(@1));
}

;

```

COMP

```

component_decl
: component_head element_list RBRACE
{
    $$ = { type: 'ComponentDecl', name: $1.name, params:
$1.params, body: $2,
           line: $1.line, col: $1.col };
}

| component_ext_head element_list RBRACE
{
    $$ = { type: 'ComponentDecl', name: $1.name, extendsFrom:
$1.extendsFrom,
           params: $1.params, body: $2, line: $1.line, col:
$1.col };
}

| component_head element_list error
{
    addSynErr(')', @3.first_line, @3.first_column+1,
'Se esperaba "]" para cerrar el componente "' + $1.name
+ '"');
}

```

```

        $$ = { type: 'ComponentDecl', name: $1.name, params:
$1.params, body: $2, error: true };
    }
    | component_bad_params_head element_list RBRACE
    {
        addSynErr($1.name, $1.line, $1.col,
            'Lista de parametros invalida en el componente "' +
$1.name + '"');
        $$ = { type: 'ComponentDecl', name: $1.name, params: [],
body: $2, error: true };
    }
;

component_head
: ID LPAREN param_list RPAREN LBRACE
{
    enterComponent($1, $3, @1.first_line, @1.first_column+1);
    $$ = { name: $1, params: $3, line: @1.first_line, col:
@1.first_column+1 };
}
;

component_ext_head
: ID KW_EXTENDS ID LPAREN param_list RPAREN LBRACE
{
    enterComponent($1, $5, @1.first_line, @1.first_column+1);
    $$ = { name: $1, extendsFrom: $3, params: $5, line:
@1.first_line, col: @1.first_column+1 };
}
;

```

SQL

```

create_stmt
: KW_TABLE IDENT KW_COLUMNS col_def_list
{ $$ = { type: 'CREATE', table: $2, columns: $4 }; }
;

```

```

update_stmt
  : IDENT LBRACK assign_list RBRACK KW_IN NUMBER
    { $$ = { type: 'UPDATE', table: $1, values: $3, id:
Number($6) }; }
  ;

```

```

delete_stmt
  : IDENT KW_DELETE NUMBER    { $$ = { type: 'DELETE', table: $1,
id: Number($3) }; }
  ;

```

Manejo de errores

El parser puede detectar dos tipos de errores: léxicos y sintácticos. La distinción se hace inyectando un `parseError` personalizado antes de cada llamada al parser:

```

parser.yy.parseError = function(msg, hash) {
  captured = {
    type:      hash.token !== undefined ? 'SINTÁCTICO' : 'LÉXICO',
    message:   msg,
    line:      hash.loc.first_line,
    col:       hash.loc.first_column,
    token:     hash.token,           // token inesperado
    expected:  hash.expected         // tokens válidos en ese punto
  };
  throw new Error(msg);
};

```


